Chapter 1

Mission Bus Integration Lab (MBIL)

1.1 Project purpose

Mission Bus Integration Lab (MBIL) is a deterministic, configuration-driven software system that simulates a simplified aerospace mission environment built around a central data bus, redundant mission computers, subsystem messaging, defined failure modes, observability, and failover behavior.

The goal is not to make a flashy simulator

The goal to demonstrate that I understand how **mission-critical systems behave, fail, reover, and get debugged.**

1.2 What MBIL should communicate

When someone sees this project, they should think:

⋆ This person understands relibable system design.

- This person thinks in interfaces and contracts
- This person knows how to model faults and degraded operation
- This person knows how to make systems observable
- This person can separate control logic, transport, config, and monitoring

MBIL should read like a **systems engineering lab**, not a game and not a toy simulation

---

1.3 Core project statement

MBIL is a **tick-driven mission-system simulation platform** with these defining properties:

- Deterministic execution
- Configuration-defined topology
- Redundant control nodes
- First-class failure scenarios

- structured observability
- Recoverable degraded operation

1.4 Design priorities

These are the real priorities, in order:

Priority 1 – Determinism The same scenario should produce the same behavior every run unless randomness is explicitly enabled

Priority 2 – Reliability modeling Failures are not side noes. They are part of the system design.

Priority 3 – Observability Every important decision, state transition, and message path should be visible in logs or snapshots.

Priority 4 – Clear modular architecture Subsystem logic, bus behavior, timing, fault injection, and loggin must stay cleanly separated.

Priority 5 – Configurability System topology and scenarios should be changel from config files, not by rewriting core logic.

1.5 Function Goals

MBIL must support:

- Subsystem-to-subsystem communication through a central bus
- Multiple sybsystem types
- Dual mission computer architecture
- Heartbeat monitoring
- Failover behavior
- Defined failure injection
- Message routing
- Deterministic Scheduling
- Structured logs
- State Snapshots
- Scenario-based runs

1.6 High-level Architecture

MBIL should be split into these major layers. **A. Simulation Core**

Responsible for:

- Tick loop
- Execution order
- Simulation Duration
- System lifecycle
- Calling each subsystem in deterministic order

B. Bus Layer Responsible for:

- Receiving Outgoing messages
- Storing pending messages
- Applying routing rules
- Applying message-related faults
- Delivering ready messages
- Preserving deterministic ordering

C. Subsystem Layer Responsible for:

- Mission Computers
- Sensors
- Display / Endpoints
- Health State
- Behavior per tick
- Message Handling

D. Fault Management Layer Responsible for:

- Defined Failure modes
- Scripted Scenario Events
- Optional Random Fault behavior later
- Recording Fault Activations
- Applying Degradation Cleanly

E. Observability Layer Responsible for:

- Structured Event Logging
- Traceable Messgage Lifecycle
- State Snapshots

- Failover Reportin
- Auditability

F. Configuration Layer Responsible for:

- Loading JSON config
- Validating Topology
- Instantiating Subsystem Objects
- Loading Scenario/Fault Definitions
- Setting timing and thresholds

1.7 Runtime Model

MBIL should use a **fixed tick** simulation model.

Example:

- One tick every 100 ms simulated time
- Run for 300 ticks per scenario

This is important because it gives you:

- Deterministic Behavior
- Clean Testability
- Easier Fault scripting
- Easier replay and debugging

Tick Processing Order
Use a Fixed order every tick:
Phase 1 – Scenario updates

- Activate any scheduled Failures
- Deactivate expired Failrues
- Apply Manual state changes if configured

Phase 2 – Sensor Updates

- Sensors decide whether to emit new data this tick

Phase 3 – Mission Computer Updates

- Active controller processes a state
- Standby Controller monitors health
- Heartbeats generated if healthy

Phase 4 – Endpoint / Display Updates

- Display or logger consumers update their local state.

Phase 5 – Bus Processing

- Pending messages sorted and processsed
- Faults applied to eligible messages
- Delayed messages retained
- Deliverable messages routed

Phase 6 – Observability Output

- Event logs Written
- Snapshot Optionally captured

That order should be documented and never casually changed.

1.8 Deterministic behavior rules

This is now a first-class design feature.

MBIL determinism rules Rule 1: fixed tick rate The simulation advances in discrete ticks.

Rule 2: stable subsystem order Subsystems are always ticked in the same deterministic order, such as sorted by ID.

Rule 3: Stable message ordering Messages created in the same tick are delivered in a defined order:

- Priority
- Tick created
- Source ID
- Message sequence number

Rule 4: No hidden asynchronous behavior in MVP No real multithreading in the initial version.

Rule 5: Reproducible scenario results A scenario file should produce identical output unless randomness is explicitly enabled with a fixed seed. This matters because it makes the system feel like an engineering lab rather than an app demo.

1.9 Core subsystem model

Each subsystem should implement a common interface.

- id()
- type()
- tick(context)
- receive(message)
- health()
- snapshot()

Health states
Every subsystem should have explicit health states such as:

- Nominal
- Degraded
- Offline
- Failed
- Recovering

This makes failure hanling more professional and easier to test.

1.10 Main subsystem types

A. Mission Computer
Two instances:

- **MC1** = primary
- **MC2** = backup

Responsibilities:

- Process incoming sensor/status messages
- Emit heartbeats
- Generate command/status output
- Maintain own health state
- Participate in failover logic

MC1 Behavior

- Starts as active controller
- Processes sensor inputs
- Emits heartbeat periodically
- Produces system status and commands

MC2 Behavior

- Starts in standby
- Watches for MC1 heartbeat loss
- Tracks timeout threshold
- Assumes control when failover condition is met
- Announces failover in log/event stream

B. Sensors
Example Sensors

- Altitude Sensor
- Temperature Sensor
- Navigation Source

Responsibilities

- Emit data on configured interval
- Send structured sensor messages
- Support degraded or failed behavior under fault scenarios

Sensor outputs should be simple but believable:

- Value
- Valid Flag
- Quality / State

C. Display/Endpoint
Responsibilities:

- Receive commands/status
- Display system state through logs or summary output
- Report currently active mission computer
- Show degraded conditions

This is not a GUI priority. Console or structured log output is engouth.

1.11 Message bus design

The bus is central, but should stay simple.

Bus responsibilities

- Accept outgoing messages from subsystems
- Assign message metadata if needed
- Hold pending and delayed queues
- Apply failure rules
- Route messages by destination
- Support broadcast
- log delivery outcome

Message routing modes

Support:

- point-to-point
- broadcast
- optional group routing later

1.12 Message model

Keep the message model explicit and interview-friendly.

Message fields

Each message should include:

- message_id
- sequence_number
- tick_created
- tick_due
- source_id
- destination_id
- message_type
- priority
- payload
- trace_id or correlation ID

That last field is very good for observability.

Message types

Starts with:

- HEARTBEAT

- SENSOR_DATA
- STATUS
- COMMAND
- ALERT
- FAILOVER_NOTICE

Payload model
Use a simple tagged payload design.
Examples:

- Heartbeat payload: current role, heatlh
- Sensor payload: sensor name, numeric value, validity
- Status payload: active controller, mode, health summary
- Command payload: command name, parameter

1.13 Failure modes as a first-class feature

This is one of the the biggest improvements.
Do not describ ehtis as "supports faults."
Describe it as:
MBIL includes defined operational failure modes and corresponding system responses.
Core failure mdoe categories
1. Bus timeout / delivery failure
A Message does not arrive in expected time.
Effect:

- Receiving system may mark source stale
- Failover logic may be triggered if heartbeats are affected

3. Message delay
Message is delivered after configured delay.
Effect:

- Stale data
- Delayed heartbeat detection
- Timing edge cases

4. Message corruption
Payload kared invalid or altered.
Effect:

- Receiving system rejects or flags data
- Degraded mode may be entered

5. Node failure
Subsystem goes offline.
Examples:

- MC1 failes
- Sensor fails

- Display goes offline

6. Node degradation
Subsystem still runs but performs poorly.
Examples:

- Lower sensor update rate
- Heartbeat jitter
- Intermittent bad data

1.14 Fault management architecture

Fault handling should not be hidden inside random subsystem code.
Create a dedicated **Fault Engine**.
Fault Engine responsibilities

- Load defined fault scenarios
- Activate faults at configured ticks
- Match faults to subsystems/messages
- Apply behavior changes
- Report activation and resolution events

Fault definition examples
Each fault should define:

- `fault_id`
- `type`
- `target`
- `start_tick`
- `end_tick` or duration
- `condition`
- `effect`
- `severity`

Example fault types in config

- subsystem offline
- drope all heartbeat messages from MC1
- delay temperature sensor messages by 3 ticks
- corrupt altitude sensor payload
- drop every third message from a source

1.15 Redundancy and failover design

This is the golden feature and should be treated seriously.

Redundancy model

- MC1 is primary
- MC2 is standby
- both may receive some information
- only one is active controller at a time

Failover trigger
MC2 should monitor MC1 heartbeat.
Example rule:

- MC1 heartbeat expected every 2 ticks
- if absent for 5 ticks, MC2 declares MC1 failed
- MC2 transitions from standby to active

Failover sequence

1. MC1 heartbeat stops
2. MC2 increments missed-heartbeat counter
3. threshold exceeded
4. MC2 logs failover event
5. MC2 becomes active controller
6. MC2 emits FAILOVER_NOTICE
7. display/status output reflects controller change

Recovery policy
For MVP, keep it simple:

- failover is one-way during a scenario
- once MC2 takes over, MC1 does not automaticlly resume control

That avoids messy arbitration logic early on... Later we can add controlled reintegration.

1.16 Observability design

This is now a major pillar of the project.

MBIL should be designed so that someone can answer:

- what happened
- when it happened
- which subsystem did it
- which message was involved
- what state the system was in before and after

Observability features
A. Structured logs
Each event logged as JSON or key-value text.
Fields should include:

- tick
- timestamp or simulated time
- category
- subsytem ID
- event name
- message ID if applicable
- trace ID if applicable
- severity
- state summary

B. Event tracing
Track key message lifecycle events:

- created
- queued
- delayed
- dropped
- delivered
- processed

C. State snapshots
At configurable intervals, output a system snaptshot:

- active controller
- subsystem health states
- latest sensor values
- queued messages
- active fautls

D. Auditability
All failover actions and fault activations should be explicitly logged.
This gives use strong language later:

- observability
- audit trail
- event traceability
- debuggability under failure

1.17 Configuration-driven architecture

This should be stronger than before.

MBIL should aim for:

System topology from config, not code.

Config should define

- subsystem list
- subsystem type
- subsystem IDs
- timing parameters
- heartbeat intervals
- timeout threshholdds
- sensor rates
- scenario duration
- active faults
- logging level
- snapshot interval

What should remain in code

Only:

- subsystem class implementations
- routing engine
- failover fules
- fault application logic

You should be able to:

- add a new sensor in JSON
- change heartbeat threshold in JSON
- switch active scenario in JSON
- run normal vs degraded modes without changing code

Setup of for senior engineer training

1.18 Suggested JSON model

System configuration

Defines structure.

```
{
   "simulation": {
      "tick_ms": 100,
      "duration_ticks": 300,
      "snapshot_interval": 10,
   },
   "bus": {
      "broadcast_id": "ALL"
   },
   "mission_control": {
      "primary_id": "MC1",
      "backup_id": "MC2",
      "heartbeat_intervale_ticks": 2,
      "failover_timeout_ticks": 5
   },
   "subsystems": [
      { "id": "MC1", "type": "mission_computer", "role": "primary" },
      { "id": "MC2", "type": "mission_computer", "role": "backup" },
      { "id": "ALT1", "type": "sensor", "sensor_type": "altitude", "rate_ticks": 2 },
      { "id": "TMP1", "type": "sensor", "sensor_type": "temperature", "rate_ticks": 5
         },
      { "id": "DSP1", "type": "display" }
   ]
}
```